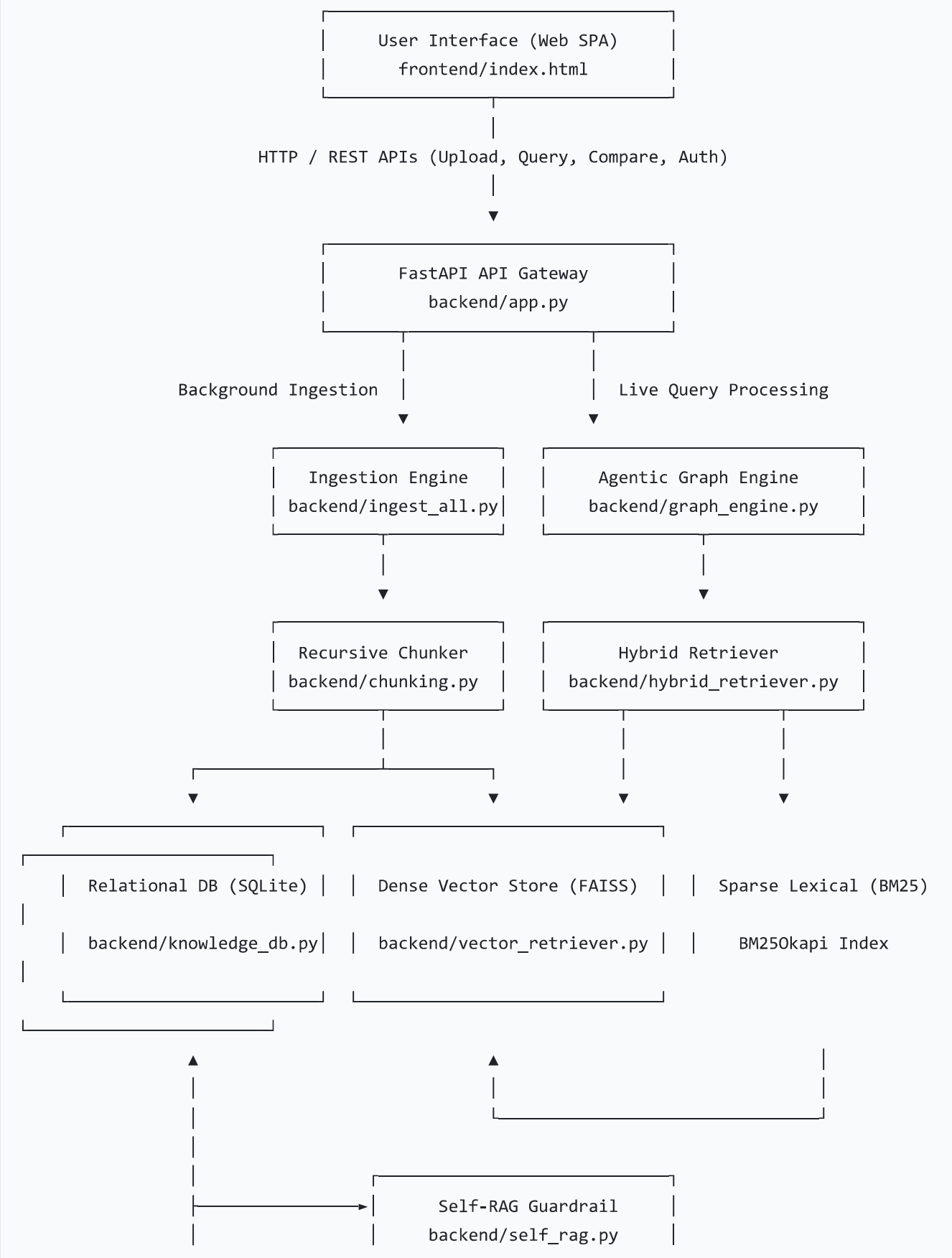
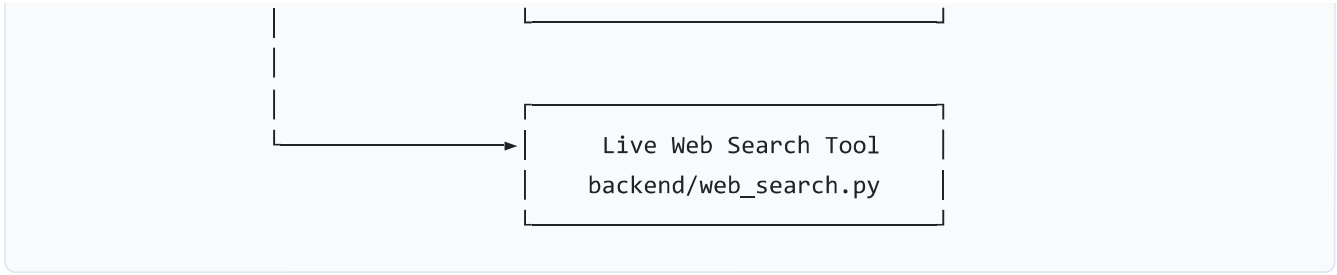
Nexus Agentic RAG System: Complete Technical Architecture & Code Guide

Executive Summary

This document provides an exhaustive, production-grade technical breakdown of the **Nexus Agentic Retrieval-Augmented Generation (RAG) System**. It details every file, class, method, data structure, and mathematical model across the backend and frontend stacks.

System Architecture Diagram





File-by-File Technical Deep Dive

1. `backend/config.py` — Central Configuration & Environment Engine

- **Purpose:** Single source of truth for all paths, model identifiers, API keys, and chunking hyperparameters.
- **Key Components:**
  - **Dynamic Path Discovery:** Uses `Path(__file__).resolve()` to anchor `BASE_DIR`, `BACKEND_DIR`, and `DOCS_DIR` relative to the script location. Eliminates OS-level path breakage.
  - **Environment Loader:** Automatically parses `.env` files from both `backend/` and root directories with a fallback raw text parser if `python-dotenv` is absent.
  - **Hyperparameters:**
    - `EMBEDDING_MODEL_NAME = "all-MiniLM-L6-v2"`: 384-dimensional dense semantic model.
    - `CHUNK_SIZE = 500`: Target character/token limit per chunk.
    - `CHUNK_OVERLAP = 100`: 20% sliding window overlap to protect sentence boundaries.

2. `backend/knowledge_db.py` — Relational Metadata & Authentication Engine

- **Purpose:** Manages SQLite relational persistence, user authentication, document tracking, and conversation history.
- **Concurrency Configuration:**
  - Uses `PRAGMA journal_mode=WAL;` (Write-Ahead Logging). Enables non-blocking concurrent reads and writes so background file ingestion never locks active chat sessions.
  - `timeout=30.0`: Prevents `database is locked` operational exceptions during parallel loads.
- **Database Tables:**

1. `users`: Stores `id`, `email`, `name`, `password_hash` (SHA-256 + salt), `role`, and `created_at`. Pre-seeds `demo@nexus.ai`.
2. `datasets`: Registry of all files, paths, MIME types, and total page counts.
3. `document_chunks`: Stores `dataset_id`, `dataset_name`, `page_number`, `chunk_index`, and raw `content`.
4. `chat_history`: Stores `user_query`, `ai_response`, `sources` (JSON array), and `agent_trace` (JSON array).

- **Core Functions:**
  - `create_user()`, `authenticate_user()`, `get_user_by_email()`: Secure salted authentication.
  - `save_chat()` & `get_compare_chats()`: Serializes and deserializes structured JSON metadata.

3. `backend/chunking.py` — Recursive Chunking & Table Preservation Engine

- **Purpose:** Slices raw document text into semantically cohesive passages while preserving structured Markdown tables and sentence integrity.
- **Core Function:** `recursive_chunk_text(text, chunk_size=500, overlap=100)`
- **Algorithm:**
  - **Hierarchical Separator Priority:** Splitting attempts descend through:
    1. Paragraph breaks (`\n\n`)
    2. Single line breaks (`\n`)
    3. Sentence terminators (`.`, `!`, `?`)
    4. Word whitespace (` `)
  - **Sliding Window:** Guarantees that key statistics or metrics straddling the 500-token threshold are preserved across contiguous chunks via a 100-character shared window.
  - **Table Defense:** Identifies Markdown pipe syntax (`| ... |`) to prevent mid-row fragmentation.

4. `backend/vector_retriever.py` — Dense Vector Search Engine (FAISS)

- **Purpose:** Vectorizes text chunks into a 384-dimensional latent space and performs nearest-neighbor semantic search.
- **Core Class:** `VectorStore` (Singleton Pattern via `__new__`)

- **Mathematical Foundations:**
  - Embedding Model: `all-MiniLM-L6-v2` (Sentence-Transformers).
  - Index Type: `faiss.IndexFlatIP(384)` (Inner Product on L2-normalized vectors). $\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\|_2 \|B\|_2} = A_{\text{norm}} \cdot B_{\text{norm}}$ Cosine Similarity(A, B) = ∥A∥2∥B∥2A·B = Anorm · Bnorm
- **Core Methods:**
  - `add_texts(chunks, metadatas, persist=True)`: Encodes chunks in batches, adds vectors to the index, appends metadata to parallel list, and writes `faiss_index.bin` + `faiss_meta.pkl`.
  - `search(query, top_k=5, filter_dataset=None)`: Normalizes query vector, searches FAISS, and applies document scope filtering if a specific file is selected.
  - `remove_document(filename)`: Reconstructs index excluding target file vectors.

5. `backend/hybrid_retriever.py` — Hybrid BM25 & Cross-Encoder Reranker

- **Purpose:** Combines sparse lexical keyword matching with dense vector search, followed by deep cross-attention reranking.
- **Core Class:** `HybridRetriever` (Singleton)
- **Pipeline:**
  1. **Sparse Lexical Search (BM25Okapi):** Evaluates exact keywords, acronyms, and codes using Inverse Document Frequency: $\text{IDF}(q_i) = \ln(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1)$IDF(qi) = ln (n(qi)+0.5N −n(qi)+0.5 + 1) Caches tokenized matrices to `bm25_cache.pkl`.
  2. **Dense Semantic Search (FAISS):** Fetches semantic concept matches.
  3. **Reciprocal Fusion & Deduplication:** Merges candidate sets and eliminates duplicate chunk IDs.
  4. **Cross-Encoder Reranking (`ms-marco-MiniLM-L-6-v2`):**
     - Unlike bi-encoders, the cross-encoder feeds `[Query, Chunk]` pairs together through full multi-head cross-attention.
     - Evaluates deep token-level interaction and assigns a rerank confidence score.
     - Selects the Top KK highest-scoring chunks.

6. `backend/web_search.py` — Live Internet Fallback Agent

- **Purpose:** Equips the agent with real-time web retrieval when local documents lack the requested information.
- **Dual-Engine Architecture:**
    1. **Primary: DuckDuckGo Search (`DDGS.text`):** Fetches real-time web search results (title, snippet, URL).
    2. **Secondary Fallback: Wikipedia OpenSearch API:** Pure Python fallback (`urllib.request`) with desktop `User-Agent` headers. Triggers if DDG encounters rate limits or CAPTCHA blocks.
- **Import Ladder:** Resilient import ladder supporting both `duckduckgo_search` and legacy `ddgs`.

7. `backend/self_rag.py` — Self-Reflective Hallucination Guardrail

- **Purpose:** Audits generated answers against source chunks to mathematically verify factual and numerical faithfulness.
- **Three Guardrail Layers:**
    1. **Stopword Filtration:** Removes 50+ syntactic noise words (`the`, `is`, `which`) so only substantive keywords contribute to grounding.
    2. **Numerical & Metric Faithfulness (`extract_numbers_and_metrics`):**
        - Regex: `r"\b\d+(?:\.\d+)?%?"`
        - Extracts percentages, integers, and decimals.
        - If an answer introduces numbers missing from the source context, it triggers an immediate **Numerical Hallucination Alert** (Score: 0.30).
    3. **Grounded Abstention:** Correctly scores honest refusals (`"NOT_IN_CONTEXT"`) as a 1.0 grounding score.

8. `backend/graph_engine.py` — Agentic Orchestrator & Synthesis Brain

- **Purpose:** Manages the complete autonomous decision loop from query arrival to final verified response.
- **Core Class:** `AgenticRAGEngine`
- **Agentic Execution Flow (`query_pipeline`):**
    1. **Query Analysis:** Deconstructs query intent and applies document scope constraint.
    2. **Hybrid Retrieval:** Executes FAISS + BM25 + Cross-Encoder pipeline.

3. **Confidence Routing:** If retrieval confidence is low, routes to `web_search.py`.
4. **Structured Table Synthesis:** Prompts Google Gemini with explicit table rules, column formatting, and citation instructions.
5. **Local Extractive Fallback:** If cloud API quotas are exhausted (HTTP 429), runs local regex/extractive pattern matching over retrieved chunks to extract accuracy and metrics without crashing.
6. **Self-RAG Verification:** Calculates final Grounding Score.
7. **Agent Trace Logging:** Assembles step-by-step reasoning timeline.

9. `backend/ingest_all.py` — Multi-Format Parsing & Batch Ingestion

- **Purpose:** Multi-modal file parser and ingestion manager.
- **Parsers:**
  - **PDF:** `extract_text_from_pdf` using `pdfplumber` + `pytesseract` OCR fallback.
  - **Tables:** `extract_tables_as_markdown` converts visual PDF grid coordinates into structured Markdown tables.
  - **Word:** `extract_text_from_docx` using `python-docx`.
  - **Spreadsheets:** `extract_text_from_csv` and `extract_text_from_excel` using `pandas` and `openpyxl`.
  - **Plain Text:** `extract_text_from_plain` for `.txt` and `.md`.
- **Smart Noise Reduction:** `is_toc_page` detects Table of Contents pages and filters them out.
- **Batch Ingestion:** `run_ingestion_pipeline()` reconciles SQLite, FAISS vectors, and BM25 tokens across the entire `Data/` directory.

10. `backend/app.py` — FastAPI Server & Asynchronous Architecture

- **Purpose:** Production API gateway, route dispatcher, and asynchronous worker manager.
- **Asynchronous Upload Architecture:**
  - **Non-Blocking Upload:** `@app.post("/upload")` writes the file to disk in $< 100\ \text{ms}$ and spawns an independent background thread (`threading.Thread(target=background_ingest_worker)`).
  - **Incremental Ingestion (`ingest_single_file`):** Parses and embeds **only the uploaded file**. Avoids wiping or re-indexing previously processed files.

    - **Thread Safety:** Employs `document_status` dictionary to track real-time progress (`processing` → `ready` → `error`).

- **Key Endpoints:**
    - `GET /` : Serves `frontend/index.html` via `FileResponse` .
    - `POST /upload` : Fast non-blocking upload endpoint.
    - `GET /upload/status/{filename}` : Real-time background status polling.
    - `POST /api/query` : Primary grounded question-answering endpoint.
    - `POST /api/compare` : Multi-document comparative analysis.
    - `GET /api/documents/preview/{filename}` : In-browser PDF stream preview.
    - `POST /api/auth/login` & `POST /api/auth/register` : User authentication.

11. `frontend/index.html` — Glassmorphic Single-Page Application (SPA)

- **Purpose:** Interactive user interface with zero external framework dependencies (Vanilla JS + Tailwind CSS + FontAwesome).
- **Core Views:**
    1. **Research Chat:** Real-time conversational interface with clickable citation cards, Grounding Score badges, and collapsible Agent Reasoning Traces.
    2. **Knowledge Library:** File management dashboard with embedded in-browser PDF preview iframe and one-click deletion.
    3. **Multi-Doc Compare:** Comparative analysis dashboard with dual file pickers.
    4. **Drag & Drop Ingestion Zone:** Modern drop zone supporting native drag-and-drop file uploads, JavaScript format validation, and automated background polling.
- **JavaScript Architecture:**
    - `sendQuery()` : Dispatches RAG queries, manages loading states, renders markdown tables and citation accordions.
    - `pollUploadStatus()` : Polls `/upload/status/{filename}` every 2 seconds until background ingestion completes.
    - `initDropZone()` : Binds `dragenter` , `dragover` , `dragleave` , `drop` , and `change` events.

Complete API Reference Table

Method	Endpoint	Request Payload	Response	Description
`GET`	`/`	None	HTML File	Serves the web application dashboard.
`POST`	`/upload`	`multipart/form-data`	`{"status": "processing", "filename": "..."}`	Saves file and triggers background indexing.
`GET`	`/upload/status/{filename}`	None	`{"status": "ready", "chunks": 6}`	Polls background indexing progress.
`POST`	`/api/query`	`{"query": "...", "active_dataset": "..."}`	Answer, Sources, Grounding Score, Agent Trace	Executes Agentic RAG pipeline.
`POST`	`/api/compare`	`{"doc a": "...", "doc b": "...", "query": "..."}`	Comparative Analysis JSON	Generates multi-document contrast report.
`GET`	`/api/documents`	None	`[{"filename": "...", "chunks": 12}, ...]`	Lists all indexed documents.
`GET`	`/api/documents/preview/{file}`	None	PDF File Stream	Streams file for in-browser PDF viewer.
`DELETE`	`/api/documents/{file}`	None	`{"status": "success"}`	Purges file from disk, SQLite, and FAISS.
`POST`	`/api/auth/login`	`{"email": "...", "password": "..."}`	User session object	Authenticates user credentials.
`POST`	`/api/auth/register`	`{"email": "...", "name": "...", "password": "..."}`	Created user record	Registers new user account.

Summary of Verification & Best Practices

1. **Model Selection:** Always configure `gemini-1.5-flash` in `backend/graph_engine.py` for genuine cloud generation.
2. **Document Management:** Individual uploads process in 1–2 seconds via background threads. Full sync (`/datasets/sync`) is only needed after manual file system edits.

3. **Security:** Never commit secret keys in `.env` files to public repositories.

Nexus Intelligent Document Knowledge Engine — Technical Documentation Complete.